

Reviewer Pack — Minimal Rank of Categorified K-theory Groups vs ACC⁰ Circuit...

Entry fbfeddc04551 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 01:29:29 UTC

Minimal Rank of Categorified K-theory Groups vs ACC⁰ Circuit Size

Entry ID: fbfeddc04551 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 01:29:29 UTC

1. Conjecture

Field A (mathematical branch): Categorification Theory **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

[‘The minimal rank of a categorified K-theory group, when applied to the symmetric monoidal category associated with an explicit function in P , is upper bounded by the size of its ACC⁰ circuit representation.’, ‘Equivalently, for any explicit function $f \in P$, there exists a categorified K-theory group G_f such that $\text{rank}(G_f) \leq s(f)$, where $s(f)$ is the smallest size of an ACC⁰ circuit computing f .’, ‘Furthermore, this bound is tight: there exist functions in P such that the minimal rank of their corresponding categorified K-theory group equals the size of their ACC⁰ circuit.’]

Rationale (proposer’s reasoning):

[‘Categorification Theory provides a framework to generalize algebraic structures and might offer new insights into computational complexity. By linking it to ACC⁰ circuits, which are known to be related to explicit functions, we could uncover non-trivial properties that separate complexity classes.’, ‘The categorified K-theory groups offer a rich source of invariants, potentially revealing deeper connections between algebraic structures and computational complexity.’, “This conjecture aims to establish a relationship that, if true, would contribute to the understanding of ACC⁰ circuits’ limitations.”]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fb36da5576b8abbe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each explicit function $f \in P$, if the minimal rank of its categorified K-theory group G_f is less than or equal to the size $s(f)$ of its ACC⁰ circuit representation, and this holds true for at least 80% of the 30 random seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - categorification theory AND ACC⁰ circuit complexity - minimal rank categorified K-theory group ACC⁰ circuit size - upper bound symmetric monoidal category ACC⁰ circuit representation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a * b) // gcd(a, b)

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(b)
        augmented_matrix = [A[i] + [b[i]] for i in range(n)]
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
```

```

        max_row = j
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
        factor = 1 / augmented_matrix[i][i]
        for j in range(n):
            augmented_matrix[i][j] *= factor
        b[i] *= factor
        for j in range(n):
            if i != j:
                factor = augmented_matrix[j][i]
                for k in range(n):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
                b[j] -= factor * b[i]
        return [row[:-1] for row in augmented_matrix], b

def rank(matrix):
    A, _ = gaussian_elimination(matrix, [0] * len(matrix))
    return sum(1 for row in A if any(row))

def construct_category(f):
    n = len(f)
    category = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            category[i][j] = f[j] - f[i]
    return category

def acc0_circuit_size(f):
    n = len(f)
    if n == 1:
        return 1
    size = n
    for i in range(1, n):
        for j in range(i+1, n):
            if f[j] - f[i] != 0:
                size += acc0_circuit_size([f[k] - f[i] for k in range(n) if k != i and k != j])
    return size

def categorified_k_theory_group(f):
    category = construct_category(f)
    return rank(category)

n = random.randint(5, 40)
f = [random.randint(1, 100) for _ in range(n)]
s_f = acc0_circuit_size(f)
G_f = categorified_k_theory_group(f)

return {
    "metric_name": "Minimal Rank of Categorified K-theory Group",
    "metric_value": G_f,
    "instances_tested": 1,
    "conjecture_holds": G_f <= s_f,
    "counterexample": "" if G_f <= s_f else f"Function: {f}, Category Size: {s_f}, Rank: {G_f}"
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Function does not satisfy the conjecture\" first")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15960
2	propose	ollama_remote	glm4:latest	0	0	14523
3	preregistration	ollama_remote	glm4:latest	0	0	9301
4	novelty	ollama_remote	glm4:latest	0	0	8456
5	novelty	ollama_remote	glm4:latest	0	0	9003
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17223

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11252
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7967
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12234
10	judge	ollama_remote	glm4:latest	0	0	12014

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117933 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/fbfeddc04551.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/fbfeddc04551.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/fbfeddc04551.tar.gz (if generated)